

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

Практика. День 9. Кол-во часов: 6.

**«АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ НА СИСТЕМУ
РЕЗЕРВНОГО КОПИРОВАНИЯ С ДЕДУПЛИКАЦИЕЙ.
ИЗУЧЕНИЕ ТРЕБОВАНИЙ К ВЗАИМОДЕЙСТВИЮ
КОМПОНЕНТОВ»**

Цель:

- Анализировать структуру и содержание технического задания (ТЗ) в соответствии с ГОСТ.
- Выделять функциональные и нефункциональные требования, особое внимание уделяя требованиям к взаимодействию компонентов.
- Определять виды интерфейсов и протоколов обмена данными между модулями программного продукта.
- Фиксировать результаты анализа в виде таблиц требований и диаграмм взаимодействия.

Выполнил: Козлов Александр,
Студент 24-ИС.

2026 г.

Вариант 8. Система резервного копирования с дедупликацией

Описание системы

Агент на клиентской машине сканирует файлы, вычисляет хэши и отправляет серверу резервного копирования список хэшей. Сервер проверяет, какие блоки уже есть в хранилище (дедупликация), и запрашивает только отсутствующие. Агент отправляет новые блоки, сервер сохраняет их в объектное хранилище (S3-совместимое). После завершения сервер обновляет метабазу (PostgreSQL).

Компоненты:

- Клиентский агент (Windows/Linux)
- Сервер резервного копирования
- Модуль дедупликации (работает с хэшами)
- Объектное хранилище (MinIO)
- Метабаза (PostgreSQL)
- Веб-панель мониторинга

Требования к взаимодействию:

- Агент ↔ Сервер: gRPC (методы CheckHashes, UploadBlocks).
- Сервер ↔ Модуль дедупликации: вызов через внутренний API (Go-канал).
- Сервер ↔ Объектное хранилище: API S3 (HTTPS).
- Сервер ↔ Метабаза: SQL через пул соединений.
- Веб-панель → Сервер: REST (получение статуса бэкапа).

Сценарий для диаграммы:

Агент отправляет хэши → сервер определяет новые блоки → агент загружает блоки → сервер сохраняет в MinIO → обновляет метабазу.

Диаграмма

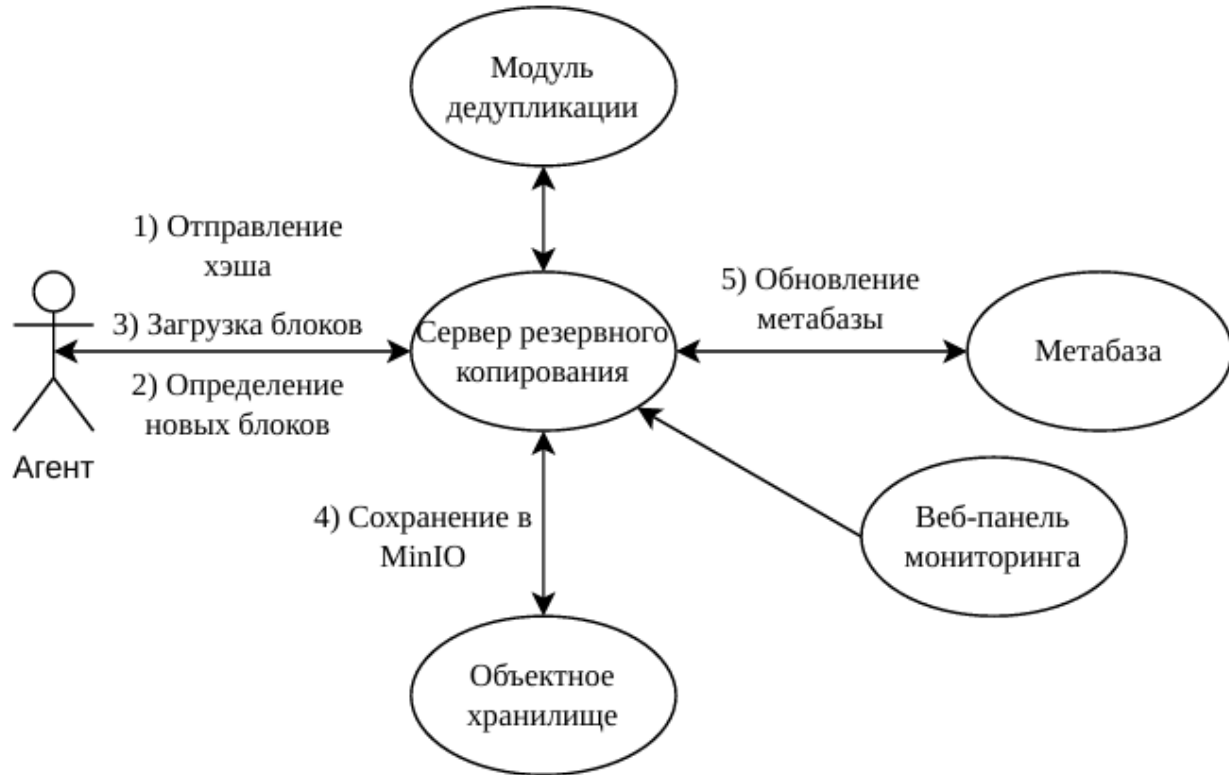


Таблица – Требования к взаимодействию компонентов

№	Компонент-источник	Компонент-приёмник	Тип взаимодействия	Протокол / API	Формат данных	Доп. Условия
1	Клиентский агент	Сервер: gRPC	↔	gRPC	JSON/XML	
2	Сервер резервного копирования	Модуль дедупликации: вызов через внутренний API; Объектное хранилище: API S3; Метабаза: SQL через пул соединений	↔		vhd, vmdk, qcow2, tar, zip, 7z	
3	Модуль дедупликации			API (Go-канал)	SHA-1, SHA-256	
4	Объектное хранилище			API S3 (HTTPS)	Content-Type	
5	Метабаза				PostgreSQL	
6	Веб-панель мониторинга	Сервер: REST	→		JSON	

Контрольные вопросы (для самопроверки)

- 1. Назовите обязательные разделы ТЗ по ГОСТ 19.201-78. В каком разделе обычно описываются требования к взаимодействию компонентов?**

Введение – наименование, область применения.

Основания для разработки – документы, на основе которых ведётся разработка.

Назначение разработки – функциональное и эксплуатационное назначение.

Требования к программе – технические, функциональные, надёжность, эргономика, **требования к взаимодействию**.

Требования к программной документации – состав и содержание документов.

Технико-экономические показатели.

Стадии и этапы разработки.

Порядок контроля и приёмки.

- 2. Чем функциональные требования отличаются от нефункциональных? Приведите пример требования к взаимодействию, которое является нефункциональным.**

Функциональные – что система должна делать (операции, сценарии).

Нефункциональные – как система это делает (производительность, безопасность, удобство).

- 3. Какие виды взаимодействия компонентов вы знаете? В каких случаях используется синхронный, а в каких – асинхронный обмен?**

Синхронное (запрос-ответ, вызов функции).

Асинхронное (события, очереди сообщений).

- 4. Что такое API? Как описать API в тексте ТЗ?**

API (Application Programming Interface): перечень операций, входные/выходные параметры, коды ошибок.

- 5. Почему важно указывать протокол и формат данных при описании взаимодействия?**

Обеспечение совместимости, гарантия корректных данных, предсказуемость, безопасность

6. Какие проблемы могут возникнуть при интеграции, если требования к взаимодействию сформулированы нечётко («модули должны обмениваться данными»)?

Если требования сформулированы нечётко, разработчик может не понять что от него требуется и не сможет продолжать делать продукт

Вывод: на мой взгляд критичных требований нет, из потенциальных проблем я могу выявить: отсутствие дополнительных требований, ограничений, единого протокола для сервера.